

Technische Universität Clausthal

Department of Informatics

Beyond Pairwise GUI Quality Judgments

Multi-Dimensional Evaluation with Modern Multimodal LLMs

Master's Thesis

Xinyang Xie

May 26, 2026

Contents

1	Introduction	1
2	Background and Related Work	2
2.1	Research Gap	2
3	Research Questions and Methodology	5
3.1	Research Questions	5
3.2	Overall Study Design	7
3.3	Track A: Screenshot-based Reproduction	7
3.4	Track B: Requirement-driven Generated Interfaces	8
3.4.1	Requirement Sources	8
3.4.2	Interface Generation	9
3.4.3	Rendering Pipeline	9
3.5	Static Evaluation Methodology	9
3.5.1	Scoring Instruments	9
3.5.2	Prompting Conditions	9
3.5.3	Scoring Safeguards	10
3.6	Dynamic Evaluation Methodology	10
3.6.1	Approach	10
3.6.2	Task Derivation	10
3.6.3	Failure Taxonomy	11
3.7	Metrics	11
3.8	Research Question – Method Map	12
A	Prompt Templates and Experimental Metadata	15
A.1	Inclusion Rule	15
A.2	Track A Pairwise Judgement Prompts	15
A.2.1	TA-PAIR-ZS-v1: Zero-shot Pairwise Visual Design Prompt . . .	15
A.2.2	TA-PAIR-RUBRIC-v1: Rubric-guided Pairwise Visual Design Prompt	16
A.3	Track B GUI Generation Prompts	17

A.3.1	Track B Generation Prompt Version History	17
A.3.2	TB-GEN-v6: Desktop Workflow-Gated HTML Generation Prompt	17
A.3.3	TB-GEN-v7: Compact Complete HTML Generation Prompt . .	20
A.3.4	TB-GEN-v8: Compact Workflow-Control-Gated Generation Prompt	22
A.4	Prompt Slots for Later Experiments	24

Chapter 1

Introduction

This chapter will introduce the motivation, research context, and contribution of the thesis. It is intentionally kept as a short placeholder so that the following draft chapters keep their intended numbering while the full introduction is developed.

Chapter 2

Background and Related Work

2.1 Research Gap

The preceding sections have reviewed GUI quality assessment, LLM-generated interfaces, LLM-as-a-judge methodology as an evaluation tool, and multimodal web agents. Table 2.1 places the most relevant prior works along seven dimensions that together characterise a comprehensive GUI quality evaluation framework. Taken together, three observations motivate the present thesis.

Existing website-generation benchmarks do not directly answer generator-leaderboard questions. UIClip [8] demonstrated that vision-language models can perform pairwise UI comparison, but it relies on models that predate the current generation of multimodal LLMs and limits evaluation to a binary preference. The LLM-as-a-judge literature [10, 4] has established scalable evaluation methods for text, yet those methods also introduce known bias and calibration concerns. In this thesis they are therefore treated as practical scoring instruments rather than as the main leaderboard target. Vision2Web [1] moves closer to this thesis by evaluating visual website development with both visual judging and agent-based verification, but its primary object is coding-agent website development rather than a thesis-specific, multi-metric leaderboard for comparing generated GUI submissions across static and dynamic metrics.

Requirement fidelity and rubric-based assessment are largely absent from GUI evaluation. Design2Code [7] evaluates how faithfully generated code reproduces a visual reference, but does not assess richer quality dimensions such as visual hierarchy, information clarity, or perceived usability. The Designer Feedback study [9] found that simple pairwise rankings among evaluators agreed at only around 50%, suggesting that coarse binary judgements are insufficient for reliable GUI assessment. A structured rubric grounded in established usability theory [5, 2] can provide finer-grained and more

reproducible quality signals.

Static visual metrics have rarely been related to dynamic interaction outcomes. VisualWebArena [3] and WebArena [11] evaluate multimodal agents on goal-directed web tasks, but do not assess the GUI quality of the interfaces themselves. How static quality scores relate to task completion, interaction feedback, and functional behaviour remains an open empirical question. Answering this question requires a benchmark that keeps generated interfaces executable so that static and dynamic evaluations can be performed on the same set of items.

The present thesis addresses all three gaps simultaneously by combining a two-track benchmark, a rubric-based scoring protocol, controlled generation metadata, validity gates, and a static-plus-dynamic leaderboard for comparing the quality of LLM-generated GUIs. Judge-model bias and human-label subjectivity are acknowledged as limitations of the scoring process, not as the central object of the leaderboard.

Table 2.1: Comparison of related work across seven evaluation dimensions. ✓ = fully addressed; △ = partially addressed; × = not addressed; — = not applicable to this line of work.

Work	Multi-dimensional rubric	Generated GUI quality	Bias and reliability analysis	Human reference labels	Requirement fidelity	Dynamic interaction eval	Benchmark artifact
UIClip [8]	×	×	×	✓	×	×	×
Designer Feedback [9]	×	✓	×	✓	△	×	×
Design2Code [7]	△	✓	×	✓	✓	×	✓
Vision2Web [1]	△	✓	△	△	✓	✓	✓
MT-Bench / Chatbot Arena [10]	△	—	△	✓	—	—	△
LLM-as-a-Judge survey [4]	✓	—	✓	—	—	—	×
Position Bias Study [6]	×	—	✓	✓	—	—	×
WebArena [11]	×	△	×	✓	△	✓	✓
VisualWebArena [3]	×	✓	×	✓	△	✓	✓
This thesis	✓	✓	✓	✓	✓	✓	✓

Chapter 3

Research Questions and Methodology

This chapter translates the motivation established in Chapter 2 into four precise research questions and describes the experimental design through which each question is answered. Section 3.1 states the research questions. Section 3.2 outlines the overall study design. Sections 3.3–3.6 detail the two dataset tracks and the static and dynamic evaluation methodologies. Section 3.7 defines all evaluation metrics. Section 3.8 maps each research question to the corresponding dataset, tasks, metrics, and expected outputs.

3.1 Research Questions

The central concern of this thesis is how the quality of LLM-generated graphical user interfaces can be compared with a reusable multi-metric benchmark and leaderboard. The primary leaderboard ranks generated GUI submissions, aggregated by generator model and requirement set. Static visual and requirement-based metrics, dynamic functionality signals, human review, and LLM-based scoring are used as measurement instruments rather than as the main objects of comparison. Four research questions operationalise this concern.

RQ1 – UIClip-style baseline and pairwise evaluation limits.

What does a reduced UIClip-style reproduction reveal about pairwise GUI-quality judgement, and what limitations motivate a shift toward a generated-GUI leaderboard with richer metrics?

This question targets the reproduction aspect of the study (Track A, Section 3.3). It uses the UIClip dataset as an anchor for pairwise screenshot evaluation, but its role is methodological and limited: to establish a baseline, identify weaknesses of single binary

preferences, and motivate the move toward richer multi-metric evaluation of generated interfaces.

RQ2 – Static multi-metric quality of LLM-generated GUIs.

How can generated GUI submissions be compared with static multi-dimensional metrics across visual structure, information clarity, and requirement fidelity?

This question is centred on Track B (Section 3.4), where different generator models produce interfaces from the same requirements, prototype screenshots, and assets. It examines whether the static portion of the leaderboard can provide interpretable per-dimension quality signals rather than only a single aggregate score.

RQ3 – Generation protocol and evaluation feasibility.

Which generation protocol, validity gates, and scoring components provide reproducible and cost-effective measurements for a generated-GUI leaderboard?

This question is answered mainly through Track B and remains a methodological supporting question. It fixes the leaderboard unit as the generated HTML submission, requires the same generation prompt version within a comparison batch, records exact generator metadata, and applies rendering and static validity gates before scoring. The LLM-as-a-judge literature [10, 4, 6] is used only to motivate caution when automated scorers are used; judge-model bias is acknowledged as a limitation rather than studied as the thesis’s main object.

RQ4 – Static and dynamic GUI quality signals.

For the same set of executable Track B interfaces, how are static quality scores related to dynamic functionality and task-validation outcomes, and where do the two kinds of metrics complement or diverge from each other?

This is the core empirical question of the thesis. Prior work has evaluated GUI quality from screenshots or evaluated agents on web tasks, but the two lines of research have not been connected on the same set of interfaces. RQ4 addresses this gap by pairing a static rubric score vector with a dynamic plan-success rate for every Track B item and analysing both correlations and mismatch cases. The goal is not to assume that static and dynamic metrics should be identical, but to understand what each signal captures about the quality of generated GUIs.

3.2 Overall Study Design

Figure 3.1 illustrates the end-to-end study design. The evaluation is organised around two complementary dataset tracks.

Track A consists of GUI screenshot pairs drawn from the UIClip dataset. Its primary function is reproduction and anchoring: by running modern multimodal LLMs on the same pairwise task that UIClip used, Track A enables a direct numerical comparison that establishes whether contemporary models represent a measurable advance. Track A items undergo static evaluation only.

Track B consists of graphical user interfaces generated from natural language requirements. Every Track B interface is kept in executable form so that it can be assessed both statically – through rubric-based scoring – and dynamically, through LLM-derived interaction plans that are validated against the rendered DOM. The key design principle is that the same interface receives both a static rubric score vector and a dynamic plan-success measurement, forming the paired data needed to answer RQ4.

The primary leaderboard unit is the generated GUI submission, aggregated by generator model and requirement set. Evaluator models, prompts, and human annotations are part of the evaluation protocol used to compute the metrics; they are not the main entities being ranked.

Human review provides an independent quality reference for selected items, but the leaderboard unit remains the generated GUI submission. Track A reuses the preference labels published with the UIClip dataset, so no new annotation is required for pairwise tasks on that track. Track B may include fresh human rubric scores for calibration and qualitative validation, collected according to the annotation protocol described in the benchmark chapter.

All experiments are documented with exact model identifiers, API version dates, prompt text, random seeds, and output files to support reproducibility.

3.3 Track A: Screenshot-based Reproduction

Track A draws on the publicly available UIClip dataset [8], which contains pairs of GUI screenshots each labelled with a human preference indicating which interface is visually superior. A random sample of 50 to 100 pairs is selected, stratified to cover the range of quality contrasts present in the original dataset.

No new human annotation is collected for Track A. The original UIClip preference labels serve as a lightweight reference for the reduced reproduction. This design makes Track A lightweight and directly comparable to the UIClip publication while keeping the main thesis contribution – a static and dynamic leaderboard for LLM-generated GUIs – concentrated in Track B.

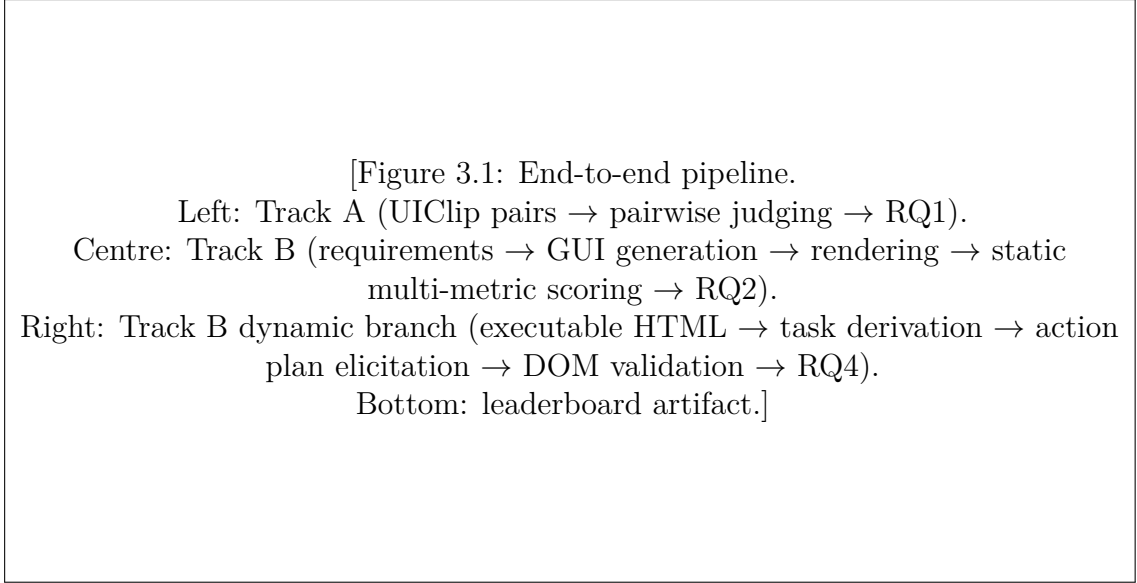


Figure 3.1: End-to-end evaluation pipeline. Track A supports reproduction (RQ1). Track B supports static multi-metric assessment (RQ2), protocol feasibility checks (RQ3), and dynamic plan validation (RQ4).

3.4 Track B: Requirement-driven Generated Interfaces

3.4.1 Requirement Sources

Track B requirements are collected from two sources. The primary source is a reduced subset of Vision2Web [1], a hierarchical benchmark for visual website development with static, interactive frontend, and full-stack levels. This thesis uses only Level 1 static webpage tasks and Level 2 interactive frontend tasks. Level 3 full-stack tasks are excluded because they introduce backend state, deployment, and long-horizon engineering requirements that are outside the scope of a controlled GUI-quality benchmark.

For each selected Vision2Web task, the provided visual prototype, structured requirement, and available assets are used as the generation input. Level 1 items support static rubric scoring; Level 2 items also support dynamic task-validation because they include interaction-oriented requirements and testable functional expectations. Approximately 10 to 20 tasks are used for the pilot phase, with a possible expansion to a small formal subset after rendering and annotation feasibility have been verified.

Design2Code [7] is retained as a static UI-to-code reference point and as a backup source for simple webpage reproduction cases, but it is no longer the primary Track B data source. A small set of author-written form-heavy requirements may be added only if the selected Vision2Web subset lacks enough clear dynamic-validation cases.

3.4.2 Interface Generation

Each requirement is submitted to two or three generator models, producing a set of generated HTML implementations per requirement. For any direct generator comparison, the same generation prompt version, input screenshots, requirement text, workflow checks, viewport policy, temperature, and output-budget policy are used wherever the provider allows it. Generator identity, provider, model identifier, date, prompt identifier, and key parameters are recorded in metadata. The leaderboard compares the quality of the generated interfaces; automated scorers and human review are used only to compute or audit the metrics.

3.4.3 Rendering Pipeline

Generated HTML files are rendered using a headless browser (Playwright) at a fixed viewport resolution. For each item, the pipeline saves a PNG screenshot and a serialised DOM snapshot. Items whose rendering produces a blank page or a JavaScript console error that prevents visible content from loading are excluded and replaced. The rendered screenshot and the DOM snapshot together constitute the input to both the static and dynamic evaluation modules.

3.5 Static Evaluation Methodology

3.5.1 Scoring Instruments

Static evaluation can combine human rubric scores, deterministic checks, and LLM-based rubric scoring. These instruments are used to score generated GUI submissions, not to create a separate evaluator leaderboard. When LLM-based scorers are used, exact model identifiers, access dates, prompt identifiers, and outputs are recorded in the reproducibility appendix.

3.5.2 Prompting Conditions

Two scoring prompt formats may be used.

Zero-shot prompting presents the model with the interface screenshot and a minimal instruction to indicate which of two interfaces is better (pairwise) or to score the interface on the rubric dimensions. This condition establishes a baseline.

Rubric-guided prompting supplies the full rubric with dimension definitions and per-level score anchors, and requires the model to return a structured JSON response containing one integer score per dimension and a one-sentence rationale. This condition tests whether explicit guidance improves alignment with the rubric definitions and produces more interpretable scoring outputs.

3.5.3 Scoring Safeguards

Two safeguards are applied where they are relevant to the scoring format.

Order-swapped pairwise comparison is used only for Track A pairwise reproduction or for optional pairwise audits. It repeats each pairwise judgement with the two interface screenshots presented in the reverse order [6]. If a model labels interface A as better when it appears first but interface B as better when it appears first, the judgement is classified as position-biased. The position-bias rate is computed per scoring model and prompting condition for audit purposes.

Repeated sampling submits selected scoring prompts three times with a non-zero sampling temperature. The self-consistency rate measures the proportion of items on which all three runs produce the same judgement.

3.6 Dynamic Evaluation Methodology

3.6.1 Approach

A computer-use agent that executes browser actions introduces two confounded sources of noise: failures caused by the interface itself and failures caused by the agent’s own planning or grounding limitations. Disentangling these sources is difficult and adds substantial engineering overhead.

Instead, this thesis uses *action plan elicitation*: an evaluator model is given the rendered screenshot, the DOM snapshot, and a short task description derived from the original requirement, and is asked to produce an ordered action plan – a JSON list of steps, each specifying an action type, a CSS or ARIA target selector, and an optional value. A deterministic validation script then checks whether each step in the plan is feasible: whether the target element exists in the DOM, whether the selector resolves to a unique element, and whether a form-submission step reaches a plausible end state. This design isolates interface quality from agent execution variability and produces a binary feasibility verdict for each action step, giving a plan-success rate per item.

Full computer-use agent execution – where an agent autonomously interacts with the live rendered page – is left as a future extension.

3.6.2 Task Derivation

One to two tasks are derived from each Track B requirement. A task is a single imperative sentence with an unambiguous success criterion that can be checked by inspecting the DOM or a screenshot of the expected end state. Example tasks include filling and submitting a contact form, selecting an item from a navigation menu, or entering a search query. Tasks are written by the author and cross-checked by one

independent reviewer to confirm clarity and feasibility.

3.6.3 Failure Taxonomy

Action plan failures are classified into four categories to support interpretable analysis of RQ4.

F1 – Interface failure: the target element is absent from the DOM, the layout prevents meaningful interaction, or a required interactive component is not implemented.

F2 – Plan failure: the element exists but the model’s selector does not resolve correctly, or the action sequence does not reach the target state despite the interface being technically functional.

F3 – Ambiguity: the task description admits multiple valid interpretations and the model’s plan addresses a different valid interpretation than the one used by the validator.

F4 – Tool failure: the validation script itself produces an error unrelated to the interface or the plan.

Cases in category F3 and F4 are reviewed manually and excluded from the RQ4 correlation analysis if they cannot be resolved.

3.7 Metrics

Track A agreement (pairwise). Cohen’s κ measures agreement between a pairwise judgement and the UIClip-derived reference label, correcting for chance. This metric is used only for the reduced Track A reproduction and optional scoring audits.

Static rubric score. The static leaderboard score is computed from per-dimension rubric scores covering visual structure, information clarity, requirement fidelity, and interaction-relevant affordances. Where both human and automated scores are available, Spearman’s ρ is reported as a calibration statistic rather than as the main leaderboard target.

Position-bias rate. The proportion of pairwise items on which the model’s preference reverses between the original and order-swapped presentation. This is reported for Track A and optional pairwise audits, not as a central Track B result.

Self-consistency rate. The proportion of items on which all three repeated-sampling runs produce the same judgement or score.

Generation validity rate. The proportion of generated submissions that pass the static validity gate: complete HTML, non-truncated script and style blocks, implemented

Table 3.1: Research question – method mapping.

RQ	Question (short)	Track	Task type	Primary metrics
1	UIClip-style pairwise baseline and evaluation limits	Track A	Pairwise judging	Accuracy vs UIClip-derived labels; position-bias rate
2	Static multi-metric quality of generated GUIs	Track B	Rubric scoring	Static rubric scores per dimension; optional human/automated calibration
3	Reproducible and cost-effective generation/evaluation protocol	Track B	Generation + rendering + scoring	Generation validity rate; render success; scorer self-consistency; API cost
4	Relationship between static and dynamic quality signals	Track B	Rubric + action plan validation	Spearman ρ (static-dynamic); plan-success rate; mismatch categories

route identifiers, defined click handlers, and clickable workflow-required controls.

Plan-success rate. The proportion of action steps in a plan that pass all three feasibility checks (existence, unique resolution, state transition). Averaged across steps within an item to obtain an item-level score.

Static-dynamic relationship. Spearman’s ρ between the item-level static rubric score (aggregate and per-dimension) and the item-level plan-success rate, computed across all Track B items. In addition, mismatch cases are reported qualitatively, because a visually strong interface can still fail a task and a visually plain interface can still be functionally successful.

API cost. Total token cost in USD per generator and scoring component, computed from provider pricing at the time of the experiment and used to characterise the feasibility trade-off addressed in RQ3.

3.8 Research Question – Method Map

Table 3.1 summarises the relationship between each research question, the dataset track and task type used to answer it, the primary metrics, and the results section that reports the findings.

Bibliography

- [1] Zehai He, Wenyi Hong, Zhen Yang, Ziyang Pan, Mingdao Liu, Xiaotao Gu, and Jie Tang. Vision2Web: A hierarchical benchmark for visual website development with agent verification, 2026. Introduces a hierarchical visual website-development benchmark with 193 tasks, 918 prototype images, and 1,255 test cases across static webpage, interactive frontend, and full-stack levels.
- [2] International Organization for Standardization. ISO 9241-11:2018 — ergonomics of human-system interaction: Usability: Definitions and concepts. International Standard 9241-11, ISO, 2018.
- [3] Jing Yu Koh, Robert Lo, Lawrence Yunliang Chen, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In *Proc. ACL*, 2024.
- [4] Dawei Li, Bohan Jiang, Liangjie Huang, et al. From generation to judgment: Opportunities and challenges of LLM-as-a-judge. In *Proc. EMNLP*, 2025.
- [5] Jakob Nielsen and Rolf Molich. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI)*, pages 249–256. ACM, 1990.
- [6] Lin Shi, Chiyu Ma, Wenhua Liang, et al. Judging the judges: A systematic study of position bias in LLM-as-a-judge. In *Proc. IJCNLP-AAACL*, 2025.
- [7] Chenglei Si, Yanzhe Zhang, Zhengyuan Yang, Ruiho Liu, Hannes Hapke, Caiming Xiong, and Diyi Yang. Design2Code: How far are we from automating front-end engineering? In *Proc. ACL*, 2024.
- [8] Jason Wu, Yi-Hao Peng, Xin Yue Amanda Li, et al. UIClip: A data-driven model for assessing user interface design. In *Proc. ACM UIST*, 2024.
- [9] Jason Wu, Amanda Swearngin, Arun Krishna Vajjala, et al. Improving user interface generation models from designer feedback. In *Proc. CHI*, 2026.
- [10] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*, 2023.

- [11] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *Proc. ICLR*, 2024.

Appendix A

Prompt Templates and Experimental Metadata

This appendix records prompts that materially affect the thesis methodology, experimental results, benchmark protocol, or reproducibility. Routine prompts used for coding, brainstorming, language polishing, or project management are not included unless their outputs become part of the experimental protocol.

A.1 Inclusion Rule

A prompt is included when changing or omitting it could change a reported metric, model comparison, benchmark construction step, annotation protocol, or reproducibility claim. For each included prompt, the experiment records the prompt text, model identifier, provider, date, key decoding parameters, input modalities, output format, random seed if applicable, and source result file.

A.2 Track A Pairwise Judgement Prompts

A.2.1 TA-PAIR-ZS-v1: Zero-shot Pairwise Visual Design Prompt

Purpose: Baseline pairwise GUI screenshot judgement for Track A, including the order-swapped reliability condition. The model receives two screenshots labelled Image A and Image B and returns only the preferred letter.

Metadata:

- **Track / task:** Track A, pairwise preference judgement
- **Prompt ID:** TA-PAIR-ZS-v1

- **Models used:** gpt-4o (OpenAI); claude-sonnet-4-5-20250929 (chatanywhere-anthropic proxy)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message
- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling); model temperature is 0 so output is deterministic
- **Order-swap:** applied; each pair is evaluated in both AB and BA orders
- **Dataset:** biglab/uiclip_human_data_hf (HuggingFace), train split
- **Source:** scripts/track_a_eval.py, lines 44–52 (ZEROSHOT_PROMPT)
- **First use:** 2026-05-06 (gpt-4o pilot); 2026-05-19 (claude-sonnet-4-5 run)
- **Result files:** scripts/results/track_a_eval_gpt-4o_20260506_*;
scripts/results/track_a_eval_chatanywhere-anthropic-claude-sonnet-4-5-20250929_*

You are evaluating two user interface screenshots for visual design quality.

Image A is shown first, Image B is shown second.

Which interface has better visual design overall? Consider layout, visual hierarchy, clarity, and aesthetic quality.

Reply with ONLY the letter A or B. Nothing else.

A.2.2 TA-PAIR-RUBRIC-v1: Rubric-guided Pairwise Visual Design Prompt

Purpose: Rubric-guided Track A pairwise GUI screenshot judgement. Adds four explicit design dimensions before requiring the same A/B output format as the zero-shot condition. Used in the prompting-strategy ablation.

Metadata:

- **Track / task:** Track A, pairwise preference judgement (rubric condition)
- **Prompt ID:** TA-PAIR-RUBRIC-v1
- **Models used:** gpt-4o (OpenAI)
- **Input modalities:** two PNG screenshots (Image A, Image B) + user-role text
- **Message structure:** single user message; no system message

- **Decoding parameters:** temperature = 0, max_tokens = 5
- **Seed:** 42 (pair sampling)
- **Order-swap:** applied
- **Dataset:** biglab/uiclip_human_data_hf (HuggingFace), train split
- **Source:** scripts/track_a_eval.py, lines 54–68 (RUBRIC_PROMPT)
- **First use:** 2026-05-19
- **Result files:** scripts/results/track_a_eval_gpt-4o_20260506_142748_pair_review/

You are an expert UI design reviewer evaluating two user interface screenshots.
Image A is shown first, Image B is shown second.

Score each interface on the following four dimensions before deciding:

1. Visual structure - layout, alignment, spacing, and visual hierarchy.
2. Information clarity - readability, labelling, and grouping of related items.
3. Aesthetic quality - typography, colour use, and overall polish.
4. Inferred usability - how easy the interface looks to use at a glance.

Briefly compare the two interfaces dimension by dimension in your head, then
pick the one that is better overall.

Reply with ONLY the letter A or B on the last line. Nothing else.

A.3 Track B GUI Generation Prompts

A.3.1 Track B Generation Prompt Version History

Track B generation prompts changed repeatedly during smoke testing. To keep the appendix focused on reproducibility rather than serving as a full prompt archive, this appendix gives the complete text for the reproducible templates currently used in smoke testing, TB-GEN-v6 and TB-GEN-v7. Earlier versions remain recoverable from source templates, generated-run metadata, and revision notes when they are needed to interpret a pilot artifact.

A.3.2 TB-GEN-v6: Desktop Workflow-Gated HTML Generation Prompt

Purpose: Current Track B GUI generation prompt for the next smoke run after reviewing the experimental scope. This version keeps desktop visual fidelity and workflow-required interactions as the hard generation target, while moving mobile/responsive quality to future work and treating unavailable PDF assets as visible-content requirements rather than real file-generation requirements.

Prompt ID	Use	Main change	Status in appendix
TB-GEN-v1	First smoke run	Baseline single-file HTML generation prompt for normalized Vision2Web pilot items.	Summarized only
TB-GEN-v2	Smoke iteration	Added explicit completion, same-file JavaScript handlers, workflow-action non-placeholder requirements, and responsive-layout constraints.	Summarized only
TB-GEN-v3-v4	Intermediate iteration	Refined workflow route handling before the fixed route-contract version.	Summarized only
TB-GEN-v5	Route-contract smoke run	Added a fixed route contract, a minimal route handler, no-scroll hash replacement, and stricter mobile overflow safeguards.	Summarized only
TB-GEN-v6	Reproducible smoke template	Made desktop workflow fidelity the hard generation target and clarified handling of unavailable downloadable assets.	Full text below
TB-GEN-v7	Compact same-prompt comparison template	Keeps the same route/workflow contract as v6 but instructs models to produce a shorter complete implementation to reduce truncation and gateway-timeout risk.	Full text below
TB-GEN-v8	Compact gated comparison template	Adds explicit exact workflow-label-to-route mappings and forbids inert elements for workflow-required controls after v7 produced an inert Local Forms control.	Full text below

Table A.1: Summary of Track B GUI generation prompt versions. Only the current templates are reproduced in full in this appendix; prior versions are retained through experiment artifacts and source history.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v6
- **Version change:** revises TB-GEN-v5 by making desktop the primary evaluation viewport, removing mobile overflow from the hard gate, allowing non-workflow secondary navigation to remain non-functional, and clarifying that missing PDF assets should be represented as visible labelled entries rather than invented files
- **Models planned/used:** qwen3.5-397b-a17b via GWDG/SAIA OpenAI-compatible API was attempted first but timed out after 360 seconds; fallback qwen3.6-35b-a3b produced the first valid GWDG artifact
- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images

- **Decoding parameters:** temperature = 0.2, max_tokens = 20000
- **Dataset:** zai-org/Vision2Web, reduced Level 1 / Level 2 pilot subset
- **Source:** scripts/generate_track_b_ui.py
- **First use:** 2026-05-25, Europe/Berlin
- **Result file:** data/track_b/items/F09_elections_bc/generated/gwdg_qwen36_35b_v6_smo

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v6

Build a single-file HTML implementation for the Track B GUI item below.

Requirements:

- Return exactly one complete HTML document, including `<!doctype html>`, `<html>`, `<head>`, and `<body>`.
- The document must end with `</html>`. Do not stop in the middle of an element, table, script, or style block.
- Put CSS and JavaScript inside the same file. Do not require a build step, package manager, backend, or network access.
- You may use local assets from the provided resource paths. The generated index.html will be saved inside:
data/track_b/items/{item_id}/generated/<run_name>/
Therefore resource paths should be referenced as `../../resources/<subpath>`.
- Match the supplied prototype screenshots as closely as possible while still implementing the written requirements.
- Implement the navigation and interactions needed by the workflow checks. Clickable navigation items must update visible
- Any function used by onclick, addEventListener, forms, menus, tabs, accordions, or navigation must be defined in the sa
- Do not use placeholder-only controls for workflow actions. If a workflow says to click "Register to Vote", "Learn more"
- Workflow-required controls must be `<a>` or `<button>` elements. Do not use inert `<div>` controls for workflow actions.
- For links not required by the workflow, plain href="#" placeholders are acceptable. For workflow-required controls, href
- Keep the implementation compact enough to finish completely. Implement the workflow-required pages and content first; s
- The primary evaluation viewport is desktop. Optimize for 1365px and 1920px wide browser rendering. Mobile/responsive qu
- If the requirement mentions downloadable files such as PDF forms but no PDF assets are provided, represent them as visi
- Secondary navigation items that are not required by the workflow may remain non-functional placeholders, but they shoul
- Include a small self-test object on window.__TRACK_B_ROUTES listing the route ids implemented for workflow navigation.
- Use the route contract below exactly. Do not invent different route ids. Every route id must appear as an element id an
- The first route in window.__TRACK_B_ROUTES is the default page shown on initial load. It must be the homepage/home rout
- Put the JavaScript route handler before `</body>`. It must define function showPage(routeId), set the selected route visi
- Prefer accessible semantic HTML, visible focus states, responsive layout, and deterministic behavior.
- Do not include explanations, markdown fences, or comments outside the HTML document.

Required route contract:

```
{route_contract}
```

Required minimal JavaScript behavior:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) {
```

```

        history.replaceState(null, '', '#' + routeId);
    }
    window.scrollTo(0, 0);
}
document.addEventListener('DOMContentLoaded', function() {
    var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
    showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>

```

```

Item ID: {item_id}
Source task: {source_task_name}
Source level: {source_level}
Use for dynamic validation: {use_for_dynamic}

```

```

Normalized requirement:
{requirement}

```

```

Workflow checks:
{workflow}

```

```

Available local resource paths:
{resource_paths}

```

```

Prototype screenshots are attached after this text in this order:
{prototype_list}

```

A.3.3 TB-GEN-v7: Compact Complete HTML Generation Prompt

Purpose: Compact Track B GUI generation prompt created after Claude TB-GEN-v6 attempts through the ChatAnywhere Anthropic-compatible endpoint failed with HTTP 524 gateway timeouts. This version keeps the same benchmark inputs and route/workflow contract, but reduces output pressure by explicitly limiting page content, tables, lists, CSS, JavaScript, and decorative detail. It is intended for same-prompt generator comparison batches; all generators in such a batch should use this prompt version.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v7
- **Version change:** revises TB-GEN-v6 by adding compactness limits, reducing non-workflow content, prioritizing completion over visual detail, and using a delayed scroll reset in the route handler
- **Models planned/used:** claude-sonnet-4-5-20250929 via ChatAnywhere Anthropic-compatible endpoint; qwen3.6-35b-a3b via GWDG/SAIA fallback for same-prompt comparison if Claude succeeds

- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** temperature = 0.2, max_tokens = 20000 for the first Claude feasibility attempt
- **Dataset:** zai-org/Vision2Web, reduced Level 1 / Level 2 pilot subset
- **Source:** scripts/generate_track_b_ui.py
- **First use:** 2026-05-26, Europe/Berlin

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v7

Build a compact, complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document, including `<!doctype html>`, `<html>`, `<head>`, and `<body>`.
- The document must end with `</html>`. Completeness is more important than decorative detail.
- Put CSS and JavaScript inside the same file. Do not require a build step, package manager, backend, or network access.
- The generated index.html will be saved inside:
data/track_b/items/{item_id}/generated/<run_name>/
Local assets must be referenced as `../../resources/<subpath>`.
- Use the supplied prototype screenshots for visual structure, spacing, typography, colors, and page hierarchy, but implement the workflow.
- Implement the workflow-required routes and interactions first. Do not spend tokens on exhaustive secondary pages.
- Workflow-required controls must be visible `<a>` or `<button>` elements and must change the displayed route/section with JavaScript.
- Secondary navigation items may be visible non-functional placeholders if the workflow does not require them.
- If downloadable files such as PDFs are mentioned but no files are provided, show labelled text links or table entries.
- Primary evaluation viewport is desktop: optimize for 1365px and 1920px. Mobile responsiveness is not a hard gate.
- Keep CSS and JS short. Avoid animations, large design systems, repeated footer markup, and long prose copied from the requirements.

Compactness limits:

- For each route, use at most one main heading, one short summary paragraph, one compact key-info block, and one table or list.
- Use at most 4 table rows per table and at most 4 bullets per list.
- Reuse shared header, sidebar, card, button, and footer styles.
- If output budget feels tight, reduce visual/detail text. Never omit closing tags, route sections, workflow buttons, or navigation items.

Route contract:

```
{route_contract}
```

Required minimal JavaScript behavior:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
};
```

```

document.querySelectorAll('[data-route-target]').forEach(function(link) {
  link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
});
if (history.replaceState) {
  history.replaceState(null, '', '#' + routeId);
}
setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>

```

```

Item ID: {item_id}
Source task: {source_task_name}
Source level: {source_level}
Use for dynamic validation: {use_for_dynamic}

```

```

Normalized requirement:
{requirement}

```

```

Workflow checks:
{workflow}

```

```

Available local resource paths:
{resource_paths}

```

```

Prototype screenshots are attached after this text in this order:
{prototype_list}

```

A.3.4 TB-GEN-v8: Compact Workflow-Control-Gated Generation Prompt

Purpose: Compact Track B GUI generation prompt created after TB-GEN-v7 successfully avoided the ChatAnywhere timeout but failed the static gate because the workflow-required Local Forms control was implemented as an inert div. This version preserves the compactness goal and adds exact workflow-label-to-route mappings plus an explicit prohibition on inert elements for workflow controls.

Metadata:

- **Track / task:** Track B, GUI generation
- **Prompt ID:** TB-GEN-v8
- **Version change:** revises TB-GEN-v7 by adding a mandatory workflow-control section with exact labels, route targets, and required `<a>` or `<button>` elements
- **Models planned/used:** claude-sonnet-4-5-20250929 via ChatAnywhere Anthropic-compatible endpoint; qwen3.6-35b-a3b via GWDG/SAIA fallback for same-prompt comparison

- **Input modalities:** normalized requirement text, Vision2Web workflow JSON, local resource-path list, route contract, and prototype screenshots
- **Message structure:** system message plus one user message containing text and prototype images
- **Decoding parameters:** temperature = 0.2, max_tokens = 20000 for the first Claude feasibility attempt
- **Dataset:** zai-org/Vision2Web, reduced Level 1 / Level 2 pilot subset
- **Source:** scripts/generate_track_b_ui.py
- **First use:** 2026-05-26, Europe/Berlin
- **First valid result:** data/track_b/items/F09_elections_bc/generated/claude_sonnet45_

System:

You are a senior frontend engineer building a benchmark submission for a GUI evaluation study.
Create a polished, working website implementation that can be opened directly from a local index.html file.

User:

Prompt ID: TB-GEN-v8

Build a compact, complete single-file HTML implementation for the Track B GUI item below.

Hard requirements:

- Return exactly one complete HTML document, including `<!doctype html>`, `<html>`, `<head>`, and `<body>`.
- The document must end with `</html>`. Completeness is more important than decorative detail.
- Put CSS and JavaScript inside the same file. Do not require a build step, package manager, backend, or network access.
- The generated index.html will be saved inside:
data/track_b/items/{item_id}/generated/<run_name>/
Local assets must be referenced as `../../resources/<subpath>`.
- Use the supplied prototype screenshots for visual structure, spacing, typography, colors, and page hierarchy, but implement workflow logic.
- Implement workflow-required routes and interactions first. Do not spend tokens on exhaustive secondary pages.
- Primary evaluation viewport is desktop: optimize for 1365px and 1920px. Mobile responsiveness is not a hard gate.
- Keep CSS and JS short. Avoid animations, large design systems, repeated footer markup, and long prose copied from the r

Mandatory workflow controls:

- Every workflow-required click target must be visible text inside an `<a>` or `<button>`. Never use `<div>`, `<span>`, `<li>`, or `<div>`.
- Each workflow control must include `data-route-target="<route_id>"` and `onclick="showPage('<route_id>')"`.
- Required exact visible labels and route targets:
 - Register to Vote -> voter_registration
 - Learn more for the 2024 Provincial Election Report banner -> 2024_provincial_election
 - Learn more for the 2026 General Local Elections card -> 2026_local_elections
 - Resources -> resources_and_reports
 - Local Forms -> local_election_forms
- If a sidebar item is workflow-required, it must still be an `<a>` or `<button>`; style it like a sidebar item with CSS instead of JS.
- Secondary navigation items may be visible non-functional placeholders only if they are not listed above.
- If downloadable files such as PDFs are mentioned but no files are provided, show labelled text links or table entries.

Compactness limits:

- For each route, use at most one main heading, one short summary paragraph, one compact key-info block, and one table or list.
- Use at most 4 table rows per table and at most 4 bullets per list.
- Reuse shared header, sidebar, card, button, and footer styles.
- If output budget feels tight, reduce visual/detail text. Never omit closing tags, route sections, mandatory workflow controls, or

Route contract:
{route_contract}

Required minimal JavaScript behavior:

```
<script>
window.__TRACK_B_ROUTES = {route_ids_json};
function showPage(routeId) {
  if (!window.__TRACK_B_ROUTES.includes(routeId)) return;
  document.querySelectorAll('[data-track-route]').forEach(function(section) {
    section.hidden = section.id !== routeId;
  });
  document.querySelectorAll('[data-route-target]').forEach(function(link) {
    link.setAttribute('aria-current', link.dataset.routeTarget === routeId ? 'page' : 'false');
  });
  if (history.replaceState) {
    history.replaceState(null, '', '#' + routeId);
  }
  setTimeout(function() { window.scrollTo(0, 0); }, 0);
}
document.addEventListener('DOMContentLoaded', function() {
  var initial = location.hash ? location.hash.slice(1) : window.__TRACK_B_ROUTES[0];
  showPage(window.__TRACK_B_ROUTES.includes(initial) ? initial : window.__TRACK_B_ROUTES[0]);
});
</script>
```

Item ID: {item_id}
Source task: {source_task_name}
Source level: {source_level}
Use for dynamic validation: {use_for_dynamic}

Normalized requirement:
{requirement}

Workflow checks:
{workflow}

Available local resource paths:
{resource_paths}

Prototype screenshots are attached after this text in this order:
{prototype_list}

A.4 Prompt Slots for Later Experiments

Future thesis-impacting prompts should be added here before or immediately after their first experimental use. Expected prompt families include Track B static rubric scoring prompts, dynamic action-plan elicitation prompts, and human annotation instructions.